

CI/CD Pipeline Setup: Git + Jenkins + Kubernetes

Overview

This document describes how to set up a CI/CD pipeline that automatically builds a Docker image and deploys it to a Kubernetes cluster whenever code is pushed to GitHub.

Flow:

git push → GitHub Webhook → Jenkins → Docker Build → Docker Hub → Kubernetes (Minikube)

Prerequisites

Tool	Version Used	Purpose
Docker Desktop	Latest	Container runtime
Git	Any	Version control
kubectl	v1.29	Kubernetes CLI
Minikube	v1.38	Local Kubernetes cluster
Jenkins	2.555.2	CI/CD automation
Cloudflare	2026.5.2	Expose Jenkins to GitHub

Step 1 — Install Tools (Windows)

Docker Desktop

Download and install from docker.com. Enable WSL2 backend during install.

Git

Download from git-scm.com. Use default settings.

kubectl

```
curl.exe -LO https://dl.k8s.io/release/v1.29.0/bin/windows/amd64/kubectl.exe
mkdir C:\kubectl
move kubectl.exe C:\kubectl\
```

Add `C:\kubectl` to System PATH via Environment Variables.

Minikube

Download installer from:

```
https://storage.googleapis.com/minikube/releases/latest/minikube-installer.exe
```

Double-click to install.

Step 2 — Start Minikube

```
minikube start --driver=docker
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	1m	v1.35.1

Step 3 — Run Jenkins via Docker

```
docker network create jenkins

docker run --name jenkins --detach `
```

```
--network jenkins `
--publish 8080:8080 --publish 50000:50000 `
--volume jenkins-data:/var/jenkins_home `
--volume /var/run/docker.sock:/var/run/docker.sock `
jenkins/jenkins:lts
```

Get initial admin password:

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Open <http://localhost:8080>, paste the password, install suggested plugins, create admin user.

Fix Docker socket permissions (required after every Jenkins restart)

```
docker exec -u root jenkins chmod 666 /var/run/docker.sock
```

Install Docker CLI inside Jenkins

```
docker exec -u root jenkins bash -c "apt-get update && apt-get install -y docker.io"
```

Install kubectl inside Jenkins

```
docker exec -u root jenkins bash -c "curl -LO https://dl.k8s.io/release/v1.29.0/bin/linux/amd64/kubectl && chmod +x kubectl && mv kubectl /usr/local/bin/"
```

Connect Jenkins to Minikube network

```
docker network connect minikube jenkins
```

Step 4 — Install Jenkins Plugins

Go to:

Manage Jenkins → Plugins → Available

Install:

- Kubernetes CLI
- Docker Pipeline
- GitHub Integration

Restart Jenkins after installing.

Step 5 — Project Files

Directory structure

```
C:\Users\Dell\OneDrive\Desktop\CICD-project\my-app\  
├─ app.sh  
├─ Dockerfile  
├─ Jenkinsfile  
└─ k8s-deployment.yaml
```

app.sh

```
#!/bin/sh  
echo "hello from my-app - v5"  
while true  
do  
    sleep 60  
done
```

The while true loop keeps the container running so Kubernetes shows Running status.

Dockerfile

```
FROM alpine
COPY app.sh .
RUN chmod +x app.sh
CMD ["/app.sh"]
```

k8s-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: vishul18/my-app:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
```

Jenkinsfile

```
pipeline {
  agent any
```

```

environment {
    IMAGE_NAME = "my-app"
    REGISTRY   = "vishul18"
}

stages {

    stage('Checkout') {
        steps {
            git branch: 'main',
                url: 'https://github.com/vishul18/CI-CD-p
project.git'
        }
    }

    stage('Build Docker Image') {
        steps {
            sh "docker build -t ${REGISTRY}/${IMAGE_NAME}:${BUILD_NUMBER} ."
            sh "docker tag ${REGISTRY}/${IMAGE_NAME}:${BUILD_NUMBER} ${REGISTRY}/${IMAGE_NAME}:latest"
        }
    }

    stage('Push to Registry') {
        steps {
            withCredentials([
                usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )
            ]) {
                sh "echo \${DOCKER_PASS} | docker login -u \${DOCKER_USER} --password-stdin"
            }
        }
    }
}

```

```

        sh "docker push ${REGISTRY}/${IMAGE_NAME}:${BUILD_NUMBER}"
        sh "docker push ${REGISTRY}/${IMAGE_NAME}:latest"
    }
}

stage('Deploy to Kubernetes') {
    steps {
        withCredentials([
            file(
                credentialsId: 'kubeconfig-creds',
                variable: 'KUBECONFIG'
            )
        ]) {
            sh "kubectl apply -f k8s-deployment.yaml"
            sh """
            kubectl set image deployment/my-app \
            my-app=${REGISTRY}/${IMAGE_NAME}:${BUILD_
NUMBER} \

            --namespace=default
            """
            sh "kubectl rollout status deployment/my-
app --namespace=default"
        }
    }
}

post {
    success { echo 'Deployment successful!' }
    failure { echo 'Pipeline failed – check logs above.' }
}

```

```
}  
}
```

Step 6 — Push to GitHub

```
cd C:\Users\Dell\OneDrive\Desktop\CICD-project\my-app  
git init  
git remote add origin https://github.com/YOUR_USERNAME/ci-cd.  
git  
git add .  
git commit -m "initial setup"  
git branch -M main  
git push -u origin main
```

Step 7 — Configure Jenkins Credentials

Go to:

Manage Jenkins → Credentials → Global → Add Credentials

Docker Hub credentials

- Kind: Username with password
- Username: your Docker Hub username
- Password: Docker Hub access token (from hub.docker.com → Account Settings → Personal Access Tokens)
- ID: dockerhub-creds

Kubeconfig credential

Generate embedded kubeconfig:


```
kubectl config view --flatten --minify > C:\Users\Dell\OneDrive\Desktop\CICD-project\my-app\kubeconfig-embedded.yaml
```

Replace 127.0.0.1 with Minikube IP:

```
# Get Minikube IP
minikube ip

# Replace in kubeconfig (example with 192.168.49.2)
(Get-Content C:\Users\Dell\OneDrive\Desktop\CICD-project\my-app\kubeconfig-embedded.yaml) -replace '127.0.0.1:PORT', '192.168.49.2' | Set-Content C:\Users\Dell\OneDrive\Desktop\CICD-project\my-app\kubeconfig-embedded.yaml
```

In Jenkins:

- Kind: Secret file
- Upload: kubeconfig-embedded.yaml
- ID: kubeconfig-creds

Step 8 — Create Jenkins Pipeline Job

1. Jenkins → New Item → name: my-app-pipeline → Pipeline → OK
2. Triggers section → check GitHub hook trigger for GITScm polling
3. Pipeline section:
 - Definition: Pipeline script from SCM
 - SCM: Git
 - Repository URL: https://github.com/YOUR_USERNAME/ci-cd.git
 - Branch: */main
 - Script Path: Jenkinsfile
4. Click Save

Step 9 — Set Up GitHub Webhook

Run Tunnel Commands

```
C:\Program Files (x86)\cloudflared\cloudflared.exe tunnel --url http://localhost:8080
```

Generate Public URL

<https://humans-today-arbor-campbell.trycloudflare.com>

- Payload URL: <https://humans-today-arbor-campbell.trycloudflare.com/github-webhook/>
- Content type: application/json
- Event: Just the push event
- Click Add webhook

Step 10 — Apply Initial Kubernetes Deployment

```
kubectl apply -f C:\Users\Dell\OneDrive\Desktop\CICD-project\my-app\k8s-deployment.yaml  
kubectl get deployments
```

Testing the Pipeline

Make a change and push:

```
cd C:\Users\Dell\OneDrive\Desktop\CICD-project\my-app  
# edit app.sh  
git add .
```

```
git commit -m "update message"
git push
```

Jenkins will auto-trigger.

Verify deployment:

```
kubectl get pods -n default
kubectl logs <pod-name> -n default
```

Monitoring & Logging Strategies for CI/CD Pipeline

Kubernetes Pod & Deployment Monitoring

Basic Commands (Manual)

```
# Check pod status
kubectl get pods -n default -w

# Check deployment rollout
kubectl rollout status deployment/my-app

# View pod logs
kubectl logs -f <pod-name> -n default

# Resource usage (needs metrics-server)
kubectl top pods -n default
kubectl top nodes
```

Enable Metrics Server in Minikube

```
minikube addons enable metrics-server  
kubectl top pods -n default
```

3. Prometheus + Grafana (Recommended Stack)

This is the industry standard for Kubernetes monitoring.

How It Works

Jenkins/K8s → Prometheus (scrapes metrics) → Grafana (visualizes)

Setup in Minikube

```
# Install using Helm  
helm repo add prometheus-community  
https://prometheus-community.github.io/helm-charts  
  
helm install monitoring prometheus-community/kube-prometheus-  
stack
```

What to Monitor

- Pod restarts — indicates crashing containers
- CPU/Memory usage — detect resource exhaustion
- Deployment replicas available — track rollout health
- HTTP request latency — if app exposes metrics

Grafana Dashboards to Use

- Kubernetes Cluster Overview
- Jenkins Performance
- Node Exporter

Common Issues & Fixes

Issue	Fix
docker: not found in Jenkins	<code>docker exec -u root jenkins bash -c "apt-get install -y docker.io"</code>
permission denied on docker.sock	<code>docker exec -u root jenkins chmod 666 /var/run/docker.sock</code>
kubectl: not found in Jenkins	Install kubectl inside Jenkins container
Minikube unreachable from Jenkins	<code>docker network connect minikube jenkins</code>
Kubeconfig uses 127.0.0.1	Replace with minikube ip in kubeconfig-embedded.yaml
Pod exec format error	Build image inside Minikube or use alpine base image
Jenkins won't start	<code>docker start jenkins</code>
Minikube context misconfigured	<code>minikube update-context</code> then <code>minikube start</code>